

Agent Take-Home Exercise

Build an Agent You'd Actually Trust

Building an agent is one thing. Knowing whether it works is another.

Build a small agentic application that a human would trust to take actions, either in a business or personally.

We want to understand:

- What should your agent do?
- How do you know when it's doing a good job?
- Where does it struggle?
- How did your evaluations (evals) help you improve it?

Pick a Problem

Choose a problem that could plausibly exist inside a real business.

A few ideas:

- Customer support triage
- HOA policy assistant
- Something else you'd actually want to build

A simple, well built, agent with thoughtful evaluations is better than a sprawling system you didn't have time to understand, so consider keeping it simple.

The Goal

Get a basic version working and run an initial evaluation.

Build at least one small evaluation dataset that covers normal cases, edge cases, and failures that would matter in the real world. Inspect

the results and traces, improve the agent, and run the evaluation again.

In short:

Build → Evaluate → Learn → Improve

What We'd Like to See

- A working agent with at least one useful tool and a retrieval step (RAG)
- At least one evaluation dataset with expected outcomes
- Deterministic checks where they make sense
- An LLM judge where human-like judgment is useful
- Traces we can look through
- Before-and-after evaluation results
- A few examples of failures, tradeoffs, or things you're still unsure about
- A short explanation of how you'd evaluate the agent in production

Perfect scores aren't the goal. We'd rather see honest failures and what you learned from them.

Nice Extras

- Calibrating an LLM judge against human-labeled examples
- Breaking results into useful slices
- Evaluating tool calls or agent trajectories
- Adding intentionally tricky examples
- Trying an online evaluation
- PII handling or guardrails
- Streaming output
- An easy setup script, Makefile, or Docker configuration

These are optional. Pick the ones that make sense for your project.

These are optional, but the ones that make sense for your project.

Tools

We use LangSmith a lot, along with LangChain, LangGraph, and Deep Agents. We'd be happy to see them here.

Other tools are welcome too. Just make sure we can see how you used them, inspect the traces and evaluations, and understand the results.

Coding assistants are welcome. Please use them. We do. Just tell us briefly how they helped and what you checked yourself.

What to Send Us

- A link to the repository
- A README with a quickstart
- A simple diagram of the agent
- Links or instructions for viewing traces and evaluation runs
- A before-and-after evaluation summary
- A short explanation of your datasets and evaluator choices
- A few known failures or limitations
- A brief screen recording

Make the main story easy to find. We'll spend about 30 minutes looking through the submission before the interview.

The Conversation

We'll spend about an hour talking through the work with you.

We'll ask about your datasets, evaluations, traces, architecture, and what you learned along the way. We may also give you a new failure case and talk through how you'd investigate it.

We're looking for good evaluation instincts, sensible architecture,

clear communication, and curiosity.

And, of course: taste, clarity, and craft.

